Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Queue operations, enqueue and dequeue

Today
Welcome to your next lesson: Queue operations, enqueue and dequeue.

I'm getting your personalized lesson ready—based on what you've already learned, what's coming next, and the best way to strengthen your skills.

Step by step, you’re moving forward.

Imagine a line of people at an ice-cream truck 🍦

New people join the back of the line.
The person at the front gets served and leaves first.

We’ll model that line with a Python list. In this lesson:

The front of the queue will be at index 0.
The rear (back) will be at the last index.

So:

Python

```
queue = ["Alice", "Ben", "Cara"]
# front -----> rear
#   0      1     2
```

New person joins the rear → we’ll use append().

Front person leaves → we’ll use pop(0).

Does this mapping make sense: index 0 = front, last index = rear?

Reply yes/no and we’ll move to a tiny code step next.

Yes, that makes sense

Nice 👍

Let’s start with the rear of the queue: adding people.

In Python, we’ll use append() to enqueue (add at the back):

Python

```
queue = []          # start with empty line
queue.append("Alice")
queue.append("Ben")
queue.append("Cara")
```

```
print(queue)
# ['Alice', 'Ben', 'Cara']
# front is index 0 → 'Alice'
# rear is last index → 'Cara'
```

So:

enqueue = add at the rear = queue.append(item)

Make sense that enqueue for our queue is just append() to the list?

If yes, I'll have you write a tiny enqueue function in the editor next.

Yes, I'm ready

Perfect

Let's turn that idea into a small, reusable function.

We'll make:

enqueue(queue, item) → adds item at the rear (back) of the list using append()

Here's your task:

Open the code editor for this lesson.

Write a function in Python:

It should be called enqueue.
It should take two parameters: queue and item.
It should use append() to add item to queue.
Optionally, you can return queue at the end (not required, but fine).
After defining it, you can test it like:

```
q = []
enqueue(q, "Alice")
enqueue(q, "Ben")
print(q)   # you should see ['Alice', 'Ben']
```

When you're happy, click Submit code so I can review it.

Go ahead and implement enqueue(queue, item) in the editor now

i adit im a bit cofused how the local scope q i s being modified withut a retur but it woks

Love that you noticed that—that's an important question

Here's the key idea, step by step:

In Python, a list is a mutable object (it can be changed in place).
When you call something like enqueue(q, "Alice"), the name q in the caller and the parameter queue inside the function both point to the same list object in memory.
Inside the function, when you do queue.append("Alice"), you're modifying that shared list object, not reassigning queue.
Because it's the same object, the change is visible through q outside the function—even without a return.

A rough mental picture:

Outside: q ⟶ [ ]
Call enqueue(q, "Alice")
Inside: queue ⟶ [ ] (same arrow, same list)
Do queue.append("Alice") → list becomes ["Alice"]
Outside: q still points to that list → now ["Alice"]
Does this “same list object, two different names” idea click, or is there a specific line from your code you want me to walk through slowly?

Python

```
def enqueue(queue, item):
    queue.append(item)


def main():
    q = []
    enqueue(q, "alice")
    enqueue(q,"ben")
    print(q)

if __name__ == "__main__":
    main()
```

Nice work Ron — that enqueue is spot on ☑

You:

Took a list q
Called enqueue(q, "alice") and enqueue(q, "ben")
Ended with ['alice', 'ben'] — exactly what we want for adding at the rear of the queue.

So you’ve got enqueue nailed. 🎯

Next step is the other side of the line: serving the front person with a dequeue that removes index 0.

Before we code it, quick check-in:

are you okay if dequeue also needs to handle the case when the queue is empty, so it doesn’t crash? (yes/no is enough)

i thin it makes sense i learne tha i a large trading report generator i made a list of a dict it you done deep copy all points to the same list or dict

Exactly 💡 That’s the same idea.

A list of dicts = many references to mutable objects.
If you don’t make a deep copy, two variables can point to the same dict or list.
Change it in one place → you see the change everywhere that points to it.

That’s why enqueue(q, "alice") works without return: inside the function you’re mutating the same list object that q refers to.

Let’s now do the other half of the queue: dequeue 🙂

Concept first, then you’ll code.

For our queue list:

Python

```
queue = ["alice", "ben", "cara"]
# front = index 0
To dequeue (serve the front person), we remove index 0:

Python

served = queue.pop(0)
# served == "alice"
# queue is now ["ben", "cara"]
So:



dequeue = remove from the front = queue.pop(0)


Does the difference between stack-style pop() (end) and queue pop(0) (front) feel clear?

If yes, I'll have you write a dequeue(queue) function next (with an empty-check).

Yes, that difference is clear

Great, then let's turn that into code 💻


We want a function:



dequeue(queue) → remove and return the front item (index 0), but only if the queue is
not empty.



Step by step behavior:


If the queue has at least one item:
remove index 0 with pop(0)
return that value
If the queue is empty:
maybe return None, or a message like "Queue is empty" (your choice)
Your task:


In the editor, write a function dequeue(queue) that:

checks if queue is non-empty using if queue:
uses queue.pop(0) when it's not empty
returns something reasonable when it is empty
Test it, for example:

Python

q = []
enqueue(q, "alice")
enqueue(q, "ben")

print(dequeue(q))  # expect 'alice'
print(q)           # expect ['ben']

print(dequeue(q))  # expect 'ben'
print(q)           # expect []

print(dequeue(q))  # now queue is empty - see what your function does
When it works, click Submit code so I can review it 👍
Python

```

```
def enqueue(queue, item):
    queue.append(item)

def dequeue(queue):
    if len(queue) > 0:
        return(queue.pop(0))
    else:
        return("Queue is empty")


def main():
    q = []
    enqueue(q, "alice")
    enqueue(q,"ben")

    print(dequeue(q))
    print(q)

    print(dequeue(q))
    print(q)

    print(dequeue(q))
    print(q)


if __name__ == "__main__":
    main()
```

This is excellent, Ron 👏

Your code does exactly what we want:

enqueue correctly adds to the rear with append.
dequeue:
Removes from the front using queue.pop(0)
Returns the removed item
Safely handles the empty queue by returning "Queue is empty"

And your output:

Text

```
alice
['ben']
ben
[]
Queue is empty
[]
```

…matches the expected behavior perfectly. You've hit both mastery goals ☑

Two small style notes (you're not wrong, just ideas):

You can use the Pythonic emptiness check:
Python

```
if queue:
    return queue.pop(0)
else:
    return "Queue is empty"
```

Sometimes people return None instead of a string when empty, but your choice is totally fine for this lesson.
To close this out, one quick check using our ice-cream line 🍦:

Imagine:

Python

queue = ["🔔Alice", "🔔Ben"]
served = queue.pop(0)
Who gets served?

I’ll put this as a one-click question next so you can choose the answer easily.

true thanks i tend to gorget shorthand , been self studying lit and dict comprehensions and generator expresions and conditional expressions as ppythons shorthand doesnt coe natural but vey hndy

Take your time with the question below 👇

If the queue is ['🔔Alice', '🔔Ben'] and we run:

served = queue.pop(0)
Who gets served?

A.

🔔Alice

B.

🔔Ben

C.

No one, the queue is empty

D.

Python will raise an error

forget

What do you think? Pick an answer in the question box above 👆

If the queue is ['🔔Alice', '🔔Ben'] and we run:

served = queue.pop(0)
Which item does served contain?

A.

'🔔Alice'

B.

'🔔Ben'

C.

'🔔Alice', '🔔Ben'

D.

It will cause an error

You nailed it, Ron 🎯

queue.pop(0) removes and returns the item at index 0, which is the front of the queue.

So with:

Python

queue = ['Alice', 'Ben']
served = queue.pop(0)
served becomes 'Alice'
queue becomes ['Ben']

That lines up perfectly with how a real line works: Alice was first, so she's served first

And I hear you on Python's "shorthand" (comprehensions, conditional expressions, etc.) — they feel weird at first but pay off big once they click. The way you've been thinking about references, deep copies, and now queues tells me you're building a very solid mental model

At this point, you've:

Implemented enqueue (add at rear with append)
Implemented dequeue (remove from front with pop(0)), with an empty check
Correctly reasoned through how the queue changes step by step

You've hit all the goals for this lesson

Ready to wrap up this lesson and move on to the next one, or do you want to poke a bit more at any of this (like the if queue: style, or more queue practice)?

Move to the next lesson

Nice work, Ron

Quick recap of what you just mastered:

You represented a queue with a Python list, with:
index 0 as the front
the last index as the rear
You implemented:
enqueue(queue, item) using queue.append(item) to add at the rear
dequeue(queue) using queue.pop(0) to remove from the front, with a safe empty check
You correctly traced what happens to the queue and the returned values step by step.
You also connected this to your own experience with lists of dicts and deep copies—that's exactly how to build deep understanding instead of just memorizing syntax

You're all set for the next lesson.
When you're ready, hit Next lesson in the Learn tab, and I'll be right there with you for the next step

```
def enqueue(queue, item):
    queue.append(item)


def main():
    q = []
    enqueue(q, "alice")
    enqueue(q,"ben")
    print(q)

if __name__ == "__main__":
    main()


def enqueue(queue, item):
    queue.append(item)

def dequeue(queue):
    if len(queue) > 0:
        return(queue.pop(0))
    else:
```

```
462             return("Queue is empty")
463
464
465     def main():
466         q = []
467         enqueue(q, "alice")
468         enqueue(q,"ben")
469
470         print(dequeue(q))
471         print(q)
472
473         print(dequeue(q))
474         print(q)
475
476         print(dequeue(q))
477         print(q)
478
479
480     if __name__ == "__main__":
481         main()
482
```